



LOGIC SIMULATOR

112705017 資財系周傑聖
112705067 資財系許云瑄



BRIEF OVERVIEW

- 📄 解析 Verilog 檔案，提取input、output、wire 以及各種邏輯閘 (AND, OR, NAND, NOR, XOR, XNOR, NOT, BUF) 的連接關係
- 📄 測試向量讀取: 讀取包含多組輸入邏輯值 (0, 1, X, Z) 的 pattern 檔案
- 📄 topological sorting: 對電路中的邏輯閘進行排序，確保每個閘的輸入值在其被求值前都已確定
- 📄 邏輯模擬: 根據topological sorting，計算每個邏輯閘的輸出值。每個邏輯閘的行為由預定義的LUT決定，考慮了 0, 1, X, Z 四種邏輯狀態
- 📄 常數處理: 能夠識別並正確處理 Verilog 中的常數輸入 1'b0 和 1'b1
- 📄 錯誤檢測: 檢測網表中的combinational loops和multi-driver nets。如果檢測到這些錯誤，程式會終止並打印相應的錯誤訊息，且不會產生輸出檔案





BRIEF OVERVIEW

Circuit - 電路的管理器

- 讀取並解析 Verilog 檔，將文本轉為電路結構。
- 存儲電路的所有組成部分，管理 Wire, Gate 的動態記憶體地址
- 執行排序，確定計算順序
- 讀取測試向量檔案
- 模擬過程，為每個測試向量調用 simulate() 方法
- 收集並格式化模擬結果，將其輸出到檔案



BRIEF OVERVIEW

Wire - 電路中的一條連線

- 內含名稱與邏輯值 ex: 0, 1, X, Z
- Circuit 類別會創建並管理所有的 Wire 物件。Gate 物件會持有指向其輸入和輸出 Wire 的指標

Gate - 邏輯閘

- Gate base class, 及 AND、OR、NOT 等邏輯閘的 inheritance class
- 每個 Gate 物件都有指向其輸入 Wire 的指標和指向輸出 Wire 的指標
- 含evaluate(), 讀取input Wire 的值, 將運算結果寫入output Wire 中, 使用 dynamic dispatch



BRIEF OVERVIEW

交互

- Circuit 解析 Verilog，創建對應的 Wire、Gate 物件，將 Gate 的輸入/輸出指標指向正確的 Wire 物件
- Circuit 進行排序，得到一個 Gate 物件的執行序列
- 對於每個測試向量，Circuit 將輸入值設置到對應的 Wire 物件上
- Circuit 按照排序遍歷 Gate 物件序列，對每個 Gate 調用其 evaluate()
- Gate::evaluate() 從其輸入 Wire 讀取值，計算後將結果寫入其輸出 Wire
 - 這個輸出 Wire 可能又是一些下游 Gate 的輸入
- 所有 Gate 計算完畢後，Circuit 從指定的 Wire 物件上讀取最終的輸出值



INHERITANCE AND POLYMORPHISM

- Base Class-Gate
- Derived Classes-AND, OR, NOR, XOR,NAND...,繼承自 Gate
- 每個Derived Classes都提供了對 evaluate() 虛擬函數的具體實現
- 應用：使用一個 `vector<Gate*>` 存儲排序後的邏輯閘，遍歷vector並呼叫 `gate->evaluate()`
- 確保調用的是指標所指向物件的正確 `evaluate()` 版本
 - 例如，gate 指向一個 AndGate 物件，則會執行 `AndGate::evaluate()` 而非 `Gate::evaluate()`

```
class Gate {
public:
    Wire* output;
    vector<Wire*> inputs;
    Gate(Wire* out_name, vector<Wire*>& input_names) : output(out_name), inputs(input_names) {}
    virtual ~Gate() = default;
    virtual void evaluate() const {
        output->value = LogicValue::UNKNOWN;
    }

    const vector<Wire*>& getInputs() const { return inputs; }
    Wire* getOutput() const { return output; }
};

class AndGate : public Gate {
public:
    AndGate(Wire* out_name, vector<Wire*>& input_names) : Gate(out_name, input_names) {}
    ~AndGate() override = default;
    void evaluate() const override;
};
```

BUGS ENCOUNTER AND SOLVE

SLIDE 6

Bug 1 : Windows 換行符 (\r\n)

- bugs: 在 Windows 環境下編輯的 Verilog 和 pattern 檔案，其換行符為 \r\n。當程式在 Linux 環境（通常只使用 \n），殘留的 \r 字元會導致字串比較失敗、正規表示式匹配異常
- solution: 在讀取整個檔案內容到字串後，進行了一個預處理步驟：使用正規表示式將所有的 \r\n 和單獨的 \r 都替換為統一的 \n

Bug 2 : 模擬結果全為 X (未知) 或輸出空白

- bugs: 模擬結果有時仍然全部是 X 或輸出格式不正確，只有 "output" 標題，沒有端口名和數據
- solution: 通過在 Circuit::build 結束時印出 input 和 output 的內容進行驗證，確保它們被正確填充。同時檢查拓樸排序後 的大小是否等於總閘數

BONUS

FUNCTIONALITIES

SLIDE 7

A

Combinational Loop Detection:

用拓樸排序的特性來檢測迴路。我們比較最終排序序列中的閘數量與電路中原始的總閘數量。如果排序序列中的閘數較少，則表明存在未被排序的閘，這些閘必然處於一個或多個組合邏輯迴路中。

B

Multi-driver Net Detection:

維護一個已記錄被驅動線路的集合。首先，將所有主要輸入線路加入此集合。然後，遍歷電路中的每一個邏輯閘，檢查其輸出線路。如果該輸出線路已經存在於被驅動線路的集合中，則說明發生了多驅動。否則，將該輸出線路加入集合。

BONUS

FUNCTIONALITIES

SLIDE 7

C

可以解析ANSI 風格 & 傳統風格的變數宣告

透過正則表達式，我們程式能夠檢測兩種宣告風格的 Verilog

```
module FA (input A, input B, input Cin, output S, output Cout);  
  
    wire P, G;  
    wire n0;  
  
    xor (P, A, B);  
    and (G, A, B);  
    xor (S, P, Cin);  
    and (n0, P, Cin);  
    or  (Cout, n0, G);  
  
endmodule
```

You, 14 小時前 • 完成 ...

```
module FA (A, B, Cin, S, Cout);  
    input A;  
    input B, Cin;  
    output S;  
    output Cout;  
  
    wire P, G;  
    wire n0;  
  
    xor (P, A, B);  
    and (G, A, B);  
    xor (S, P, Cin);  
    and (n0, P, Cin);  
    or  (Cout, n0, G);  
  
endmodule
```

You, 14 小時前 •

BONUS

FUNCTIONALITIES

SLIDE 7



專門保管動態記憶體

透過Circuit 物件中的 m_wires_map, m_gates_storage 一個 map 一個 vector 來負責管理所有動態記憶體的地址，當程式要結束時，destructor 只需遍歷他們，並 delete 即可，不會發生記憶體管理失敗的問題

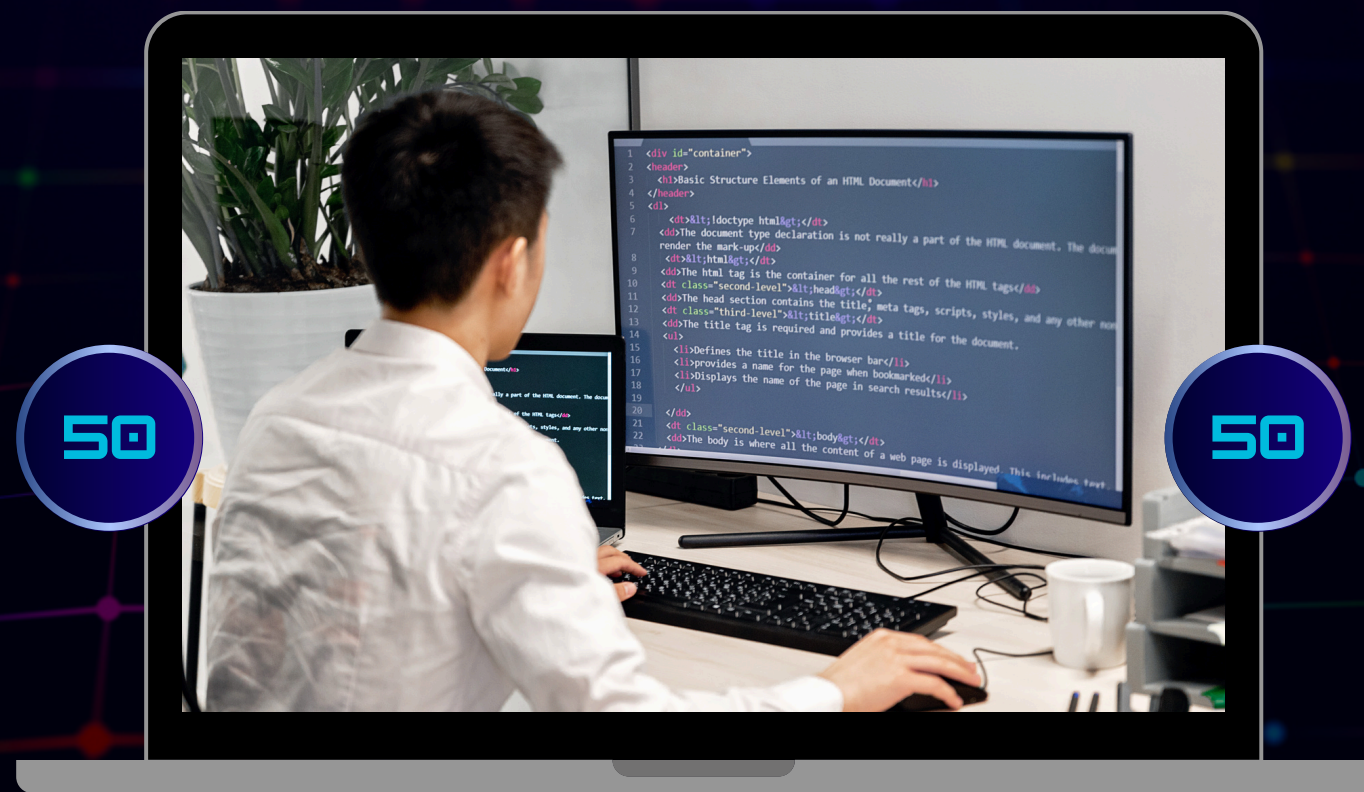
```
~Circuit() {  
    for (auto& gate : m_gates_storage) {  
        delete gate;  
    }  
    for (auto& pair : m_wires_map) {  
        delete pair.second;  
    }  
}
```



WORK

許云瑄

- Verilog 檔案讀取與預處理
- Module Header 解析
- 邏輯閘聲明解析
- Wire 物件管理與創建
- Gate 物件的初步資訊收集與創建
- 多驅動網路檢測邏輯
- 做報告



周傑聖

- Gate 基底類別與衍生類別的設計與實現
- 拓樸排序演算法實現
- 組合邏輯迴路檢測邏輯
- Pattern 檔案讀取與解析
- 模擬執行流程
- 組合邏輯迴路檢測邏輯
- 報告



FEEDBACK

在這一次的作業中，我們不僅將數位邏輯的理論知識成功轉化為實際可運行的程式碼，也實際運用了上課所學的東西，像是inheritance and polymorphism及topological sorting。開發過程中，我們體會到正規表示式在解析Verilog時的強大與複雜性，並認識到環境的換行符差異、字串處理細節等對程式正確性的重要影響。面對bug時，我們會透過輸出錯誤訊息來找到問題點，提升了問題解決能力。

在團隊合作方面，我們遇到問題會互相幫助，積極溝通，共同討論架構與疑難點，克服困難。

最後成功實現了預期功能及額外錯誤檢測，不僅是一次檢驗整學期所學的內容，也鍛煉了我們解決複雜問題和團隊協作的 ability。



THANK
YOU!